

Deployment Guide

Complete step-by-step guide for deploying MergeTasks to OVH VPS

Version 1.0 — April 2026

Prepared for the MergeTasks engineering team

Confidential — Do not distribute

Table of Contents

01	Prerequisites
02	Server Preparation (OVH)
03	Install System Dependencies
04	Database Setup (MySQL)
05	Redis Setup
06	Upload and Install Application
07	Environment Configuration
08	Run Database Migrations
09	Build the Application
10	Verify SMTP Before Going Live
11	Run Integration Tests
12	Process Management (PM2)
13	DNS Configuration (GoDaddy)
14	Nginx Reverse Proxy + SSL
15	Cloudflare Setup
16	Post-Deploy Verification Checklist
17	Monitoring and Health Checks
18	Platform Admin Dashboard
19	SSO Configuration
20	Updating to a New Version
21	Troubleshooting
22	Environment Variables Reference
23	Migration Files Reference

01 — Prerequisites

Before you begin, confirm you have all of the following. Do not proceed until every item is checked.

Requirement	Details
OVH VPS	Ubuntu 22.04 LTS, minimum 2 vCPU / 4 GB RAM / 40 GB SSD
Domain	<code>mergetasks.com</code> on GoDaddy (or your registrar)
SSH access	Root or sudo user on the OVH VPS
Source code	The latest <code>mergetasks-*.zip</code> archive
Gmail App Password	For SMTP. Generate at myaccount.google.com/apppasswords
OpenAI API Key	For AI copilot. Get from platform.openai.com/api-keys
Stripe Keys (optional)	For subscription billing. Get from dashboard.stripe.com/apikeys
Cloudflare Account	Free tier. For CDN, DDoS protection, and WAF

Critical Warning

Login requires working SMTP. If email delivery fails, users cannot receive verification codes and cannot sign in. Do not skip the SMTP verification step (Section 10).

02 — Server Preparation (OVH)

Connect to your server via SSH:

```
ssh root@YOUR_OVH_IP
```

Create a deploy user. Do not run the application as root:

```
adduser mergetasks
usermod -aG sudo mergetasks
su - mergetasks
```

Update the system:

```
sudo apt update && sudo apt upgrade -y
```

03 — Install System Dependencies

Install Node.js 22.x, pnpm, MySQL, Redis, Nginx, and Certbot:

```
# Node.js 22.x
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Verify
node --version # should print v22.x.x

# pnpm
sudo npm install -g pnpm

# MySQL 8
sudo apt install -y mysql-server

# Redis
sudo apt install -y redis-server
sudo systemctl enable redis-server
sudo systemctl start redis-server

# Verify Redis
redis-cli ping # should print PONG

# Nginx
sudo apt install -y nginx

# Certbot (SSL)
sudo apt install -y certbot python3-certbot-nginx

# PM2 (process manager)
sudo npm install -g pm2
```

04 — Database Setup (MySQL)

Secure MySQL and create the application database:

```
sudo mysql_secure_installation
```

Then create the database and user:

```
sudo mysql

CREATE DATABASE mergetasks CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'mergetasks'@'localhost' IDENTIFIED BY 'GENERATE_A_STRONG_PASSWORD_HERE';
GRANT ALL PRIVILEGES ON mergetasks.* TO 'mergetasks'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Save this password.

You will need it for `DATABASE_URL` in the `.env` file.

Test the connection:

```
mysql -u mergetasks -p mergetasks -e "SELECT 1;"
```

05 — Redis Setup

Redis should already be running from step 3. Verify:

```
redis-cli ping  
# Expected: PONG
```

To password-protect Redis (recommended for production):

```
sudo nano /etc/redis/redis.conf  
# Find and set: requirepass YOUR_REDIS_PASSWORD  
  
sudo systemctl restart redis-server
```

Your `REDIS_URL` will be: `redis://:YOUR_REDIS_PASSWORD@localhost:6379`

If no password: `redis://localhost:6379`

06 — Upload and Install Application

From your local machine, upload the source archive:

```
scp mergetasks-*.zip mergetasks@YOUR_OVH_IP:~/
```

On the server:

```
cd ~  
unzip mergetasks-*.zip -d mergetasks-app  
cd mergetasks-app
```

Install dependencies:

```
pnpm install
```

This will take 2–5 minutes. Verify it completes without errors. Then create the uploads directory:

```
mkdir -p uploads
```

07 — Environment Configuration

Copy the example and fill in every required value:

```
cp .env.example .env
nano .env
```

You must set every variable marked [REQUIRED].

The `.env.example` file documents all variables with clear labels. Do not guess — if a value is missing, the application will fail silently or crash.

Minimum viable configuration:

```
# App
NODE_ENV=production
VITE_APP_ID=mergetasks
PORT=3000
APP_BASE_URL=https://app.mergetasks.com
APP_URL=https://app.mergetasks.com
ALLOWED_ORIGINS=https://app.mergetasks.com,https://mergetasks.com

# Database
DATABASE_URL=mysql://mergetasks:YOUR_DB_PASSWORD@localhost:3306/mergetasks

# Auth secrets — generate both with:
# node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
SESSION_SECRET=PASTE_GENERATED_SECRET_HERE
CREDENTIAL_ENCRYPTION_KEY=PASTE_GENERATED_KEY_HERE

# SMTP — REQUIRED FOR LOGIN
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-gmail-app-password
SMTP_FROM=your-email@gmail.com

# OpenAI
OPENAI_API_KEY=sk- ...

# Redis
REDIS_URL=redis://localhost:6379
```

Generate the secrets now:

```
node -e "console.log('SESSION_SECRET=' + require('crypto').randomBytes(48).toString('hex'))"
node -e "console.log('CREDENTIAL_ENCRYPTION_KEY=' + require('crypto').randomBytes(32).toString('hex'))"
```

08 — Run Database Migrations

The migrations must be run in order. There are 32 migration files. Push the base schema using Drizzle, then run the three newest migrations manually:

```
# Push the base schema (handles migrations 0000-0028 automatically)
pnpm db:push

# Then run the three new migrations manually:
mysql -u mergetasks -p mergetasks < drizzle/0029_pci_lockout_and_audit_fix.sql
mysql -u mergetasks -p mergetasks < drizzle/0030_complete_foreign_keys.sql
mysql -u mergetasks -p mergetasks < drizzle/0031_store_identity_providers.sql
```

Migration	Purpose
0029	Account lockout columns, OTP hash widening (6→64 chars), audit timestamp VARCHAR→DATETIME(3)
0030	32 missing FK constraints with proper CASCADE/SET NULL behavior
0031	SSO: store_identity_providers table + ssoProviderId / ssoSubject on store_users

Verify the migrations applied:

```
mysql -u mergetasks -p mergetasks -e "SHOW COLUMNS FROM users LIKE 'failedLoginAttempts';"
# Should return one row

mysql -u mergetasks -p mergetasks -e "SHOW TABLES LIKE 'store_identity_providers';"
# Should return one row
```

09 — Build the Application

```
pnpm build
```

This runs two steps: `vite build` compiles the React frontend into `dist/client/`, and `esbuild` bundles the Express server into `dist/index.js`.

Verify the build succeeded:

```
ls -la dist/index.js
# Should exist and be > 1 MB
```


10 — Verify SMTP Before Going Live

This is the most important pre-launch check.

If SMTP does not work, nobody can log in. The application deliberately blocks login when email delivery fails — there is no fallback.

Run the built-in SMTP connectivity test:

```
node -e "
const nodemailer = require('nodemailer');
const t = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp.gmail.com',
  port: parseInt(process.env.SMTP_PORT || '587'),
  secure: false,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
});
t.verify()
  .then(() => console.log('SMTP OK'))
  .catch(e => console.error('SMTP FAILED:', e.message));
"
```

Expected output: SMTP OK

Then send a real test email:

```
node -e "
const nodemailer = require('nodemailer');
const t = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp.gmail.com',
  port: parseInt(process.env.SMTP_PORT || '587'),
  secure: false,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
});
t.sendMail({
  from: process.env.SMTP_FROM,
  to: process.env.SMTP_USER,
  subject: 'MergeTasks SMTP Test',
  text: 'If you see this, SMTP is working.'
}).then(info => console.log('Sent:', info.messageId))
  .catch(e => console.error('FAILED:', e.message));
"
```

Check your inbox. If the email arrives, SMTP is confirmed working. Proceed.

If SMTP fails

- Verify your Gmail App Password is correct (not your regular password)
- Verify 2-Step Verification is enabled on the Gmail account
- Check that port 587 is not blocked by OVH firewall
- Try port 465 with `secure: true` if 587 is blocked

11 — Run Integration Tests

The project includes 54 integration tests covering all critical write paths. Run them before going live to confirm the security and data integrity features are working correctly.

```
pnpm test
```

All 54 tests should pass. The test suite covers:

Test Group	Count	What It Validates
Password Policy	8	12-char minimum, uppercase, lowercase, digit, special char enforcement
OTP Hashing	5	SHA-256 hashing produces correct 64-char hex, deterministic output
Account Lockout	6	Lockout after 10 failures, 30-min window, reset on success
Token Blocklist	6	Session revocation, TTL expiry, concurrent access safety
PKCE Store	5	Server-side code verifier storage, one-time consumption, TTL cleanup
Data Retention	3	Cleanup function export, table references, retention periods
Audit Logging	4	All SSO action types registered, timestamp format
SSO Utilities	4	SAML/OIDC provider exports, user resolver JIT logic
Email Templates	3	SSO onboarding email HTML generation, branded content
Rate Limiting	3	Store public endpoint rate limit configuration
Platform Admin	3	Admin-only access enforcement, router registration
Health Endpoint	2	DB and Redis ping, degraded status on failure
Encryption	2	Credential encryption round-trip, key validation

If any test fails

, do not deploy. The test name will indicate which security feature is broken. Fix the issue and re-run before proceeding.

12 — Process Management (PM2)

Start the application with PM2:

```
pm2 start dist/index.js --name "mergetasks" --env production
```

Verify it is running:

```
pm2 status
# Should show "mergetasks" with status "online"

pm2 logs mergetasks --lines 20
# Should show startup logs without errors
```

Configure PM2 to start on boot:

```
pm2 save
pm2 startup
# Follow the printed command (copy-paste and run it)
```

13 — DNS Configuration (GoDaddy)

Log into your GoDaddy account and navigate to DNS management for `mergetasks.com`.

Do NOT modify any existing MX, TXT, SRV, or CNAME records
related to Outlook/Microsoft 365. Those control your `info@mergetasks.com` email.

Add this record:

Type	Name	Value	TTL
A	app	YOUR_OVH_IP	1 Hour

This creates `app.mergetasks.com` pointing to your server. Verify DNS propagation (may take 5–30 minutes):

```
dig app.mergetasks.com +short
# Should return your OVH IP
```

14 — Nginx Reverse Proxy + SSL

Create the Nginx configuration:

```
sudo nano /etc/nginx/sites-available/mergetasks
```

Paste this configuration:

```
server {
    server_name app.mergetasks.com;

    client_max_body_size 50M;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
        proxy_send_timeout 120s;
    }

    location /uploads/ {
        alias /home/mergetasks/mergetasks-app/uploads/;
        expires 30d;
        add_header Cache-Control "public, no-transform";
    }

    location = /health {
        proxy_pass http://localhost:3000/health;
        proxy_set_header Host $host;
    }
}
```

Enable the site and get SSL:

```
# Enable
sudo ln -s /etc/nginx/sites-available/mergetasks /etc/nginx/sites-enabled/

# Remove default site
sudo rm -f /etc/nginx/sites-enabled/default

# Test config
sudo nginx -t

# Restart
sudo systemctl restart nginx

# Get SSL certificate
sudo certbot --nginx -d app.mergetasks.com
```

Certbot will automatically update the Nginx config to handle HTTPS and redirect HTTP to HTTPS.

15 — Cloudflare Setup

Cloudflare provides DDoS protection, edge caching, WAF (Web Application Firewall), and SSL termination. The free tier is sufficient for launch.

15.1 — Add Your Domain

1. Sign up at `dash.cloudflare.com`
2. Click "Add a Site" → enter `mergetasks.com`
3. Select the **Free** plan
4. Cloudflare will scan your existing DNS records. Verify all records are imported correctly, especially MX records for email
5. Cloudflare will provide two nameservers (e.g., `ada.ns.cloudflare.com` , `bob.ns.cloudflare.com`)

15.2 — Update Nameservers at GoDaddy

1. Go to GoDaddy → Domain Settings → Nameservers
2. Change from GoDaddy nameservers to the two Cloudflare nameservers
3. Save. Propagation takes 1–24 hours (usually under 1 hour)

Important:

After switching nameservers, all DNS management moves to Cloudflare. Any future DNS changes must be made in the Cloudflare dashboard, not GoDaddy.

15.3 — DNS Records in Cloudflare

Verify these records exist in Cloudflare DNS (they should have been imported):

Type	Name	Value	Proxy
A	<code>app</code>	<code>YOUR_OVH_IP</code>	Proxied (orange cloud)
MX	<code>@</code>	(your existing mail records)	DNS only (grey cloud)
TXT	<code>@</code>	(SPF, DKIM, etc.)	DNS only

MX records must NOT be proxied.

Email records (MX, SPF, DKIM, DMARC) must always be "DNS only" (grey cloud). Proxying them will break email delivery.

15.4 — SSL/TLS Settings

1. Go to SSL/TLS → Overview
2. Set encryption mode to **Full (strict)** — this requires a valid SSL cert on your origin (Certbot provides this)

3. Go to SSL/TLS → Edge Certificates → enable "Always Use HTTPS"
4. Enable "Automatic HTTPS Rewrites"
5. Set Minimum TLS Version to **TLS 1.2**

15.5 — Caching Rules

Go to Caching → Cache Rules and create these rules:

Rule Name	Match	Action
Cache static assets	URI path contains <code>/assets/</code>	Cache Everything, Edge TTL: 1 month
Cache uploaded files	URI path starts with <code>/uploads/</code>	Cache Everything, Edge TTL: 7 days
Bypass API cache	URI path starts with <code>/api/</code>	Bypass Cache
Bypass health check	URI path equals <code>/health</code>	Bypass Cache

15.6 — WAF (Security) Settings

1. Go to Security → WAF
2. Enable the **Cloudflare Managed Ruleset** (free tier includes basic rules)
3. Set Security Level to **Medium**
4. Enable Bot Fight Mode (Security → Bots)

15.7 — Rate Limiting (Optional)

The application already has server-side rate limiting on auth and public store endpoints. Cloudflare's free tier includes 1 rate limiting rule. Consider adding:

Rule	Match	Threshold	Action
Auth abuse	URI path starts with <code>/api/trpc/storeAuth</code>	20 requests per 10 seconds per IP	Block for 1 hour

15.8 — Verify Cloudflare is Working

```
curl -I https://app.mergetasks.com
# Look for: cf-ray header (confirms Cloudflare is proxying)
# Look for: server: cloudflare
```

16 — Post-Deploy Verification Checklist

Run through every item. Do not skip any. Check each box as you complete it.

- ☐ **Health check** — `curl https://app.mergetasks.com/health` returns `{"status": "healthy"}`
- ☐ **Homepage loads** — Open `https://app.mergetasks.com` in browser, sign-in page renders
- ☐ **Sign up** — Create a new account, verification code email arrives within 30 seconds
- ☐ **Sign in** — Enter the code from email, redirected to dashboard
- ☐ **Password policy** — Try signing up with "abc123", rejected with complexity requirements
- ☐ **Account logout** — Enter wrong password 10 times, "Account temporarily locked" message
- ☐ **Create a client** — Dashboard → Clients → Add, client saved
- ☐ **Create a store** — Dashboard → Stores → Create, store created with slug
- ☐ **Store public page** — Visit `https://app.mergetasks.com/s/YOUR-SLUG`, store renders publicly
- ☐ **AI copilot** — Type "Show me my clients" in dashboard chat, AI responds
- ☐ **Cloudflare** — `curl -I https://app.mergetasks.com` shows `cf-ray` header
- ☐ **SSL** — Browser shows padlock icon, no mixed content warnings
- ☐ **Admin console** — Navigate to `/platform-admin`, shows metrics (admin only)

17 — Monitoring and Health Checks

The `/health` endpoint pings both MySQL and Redis and returns:

```
{
  "status": "healthy",
  "checks": {
    "database": "ok",
    "redis": "ok"
  }
}
```

If either service is down, it returns HTTP 503 with `"status": "degraded"`.

Set up uptime monitoring

Use any of these free services to monitor `https://app.mergetasks.com/health` every 60 seconds:

- **UptimeRobot** — free for 50 monitors
- **Better Uptime** — free tier available
- **Cronitor** — free for 5 monitors

PM2 monitoring

```
# View real-time logs
pm2 logs mergetasks

# View process metrics
pm2 monit

# Restart if needed
pm2 restart mergetasks
```

Automated data retention runs every 6 hours after server startup. It cleans up expired verification codes, used/expired password reset tokens, old notifications, and expired store approval tokens. No cron job setup is needed.

18 — Platform Admin Dashboard

The platform owner has access to `/platform-admin` — a dashboard visible only to users with `role: "admin"`.

Metric	Description
Total Users	All registered accounts
Signups (24h / 7d / 30d)	New registrations over time
Active Users (7d / 30d)	Users who signed in recently
Churned (30d)	Users active 30–60 days ago but not in the last 30 days
MRR	Monthly recurring revenue from paying subscribers
Tier Breakdown	Free / Pro / Enterprise counts with active/canceled/past-due status
Suspend/Unsuspend	Immediately revoke all sessions and lock any user account

To set yourself as admin (first-time setup):

```
mysql -u mergetasks -p mergetasks -e "UPDATE users SET role = 'admin' WHERE id = 1;"
```

Sign out and sign back in. The "Admin Console" link will appear in the sidebar.

19 — SSO Configuration

MergeTasks supports enterprise SSO via SAML 2.0 and OIDC for store (end-user) authentication. SSO is configured per-store by the distributor, and is available on the Enterprise plan.

How it works

1. Distributor goes to Store Editor → SSO tab
2. Adds an Identity Provider (Okta, Azure AD, Google Workspace, etc.)
3. Configures the IdP with the SP metadata URL provided by MergeTasks
4. End users type their email on the store login page — if the email domain matches a configured IdP, they are redirected to SSO automatically
5. Users who don't match any SSO domain fall through to the standard OTP/password flow

Key URLs

Endpoint	Purpose
<code>/api/sso/saml/metadata/:storeId</code>	SP metadata XML (give this to the IT admin configuring the IdP)
<code>/api/sso/saml/callback/:storeSlug</code>	SAML Assertion Consumer Service (ACS) URL
<code>/api/sso/oidc/callback/:storeSlug</code>	OIDC redirect URI

Testing with Okta Dev

1. Create a free Okta developer account at `developer.okta.com`
2. Create a new SAML 2.0 application
3. Set the Single Sign-On URL to `https://app.mergetasks.com/api/sso/saml/callback/YOUR-STORE-SLUG`
4. Set the Audience URI to `https://app.mergetasks.com/api/sso/saml/metadata/YOUR-STORE-ID`
5. Download the IdP metadata XML or copy the SSO URL and X.509 certificate
6. In MergeTasks Store Editor → SSO → Add Provider → paste the values
7. Assign a test user in Okta and try logging in

SSO Setup Guide for IT Admins:

A branded PDF guide is included at `client/public/guides/MergeTasks_SSO_Setup_Guide.pdf`. It is automatically sent as an email attachment when a distributor creates a new IdP configuration. Distributors can also download it from the SSO settings page.

20 — Updating to a New Version

When you receive a new source archive:

```
# On the server
cd ~/mergetasks-app

# Back up the current version
cp -r . ../mergetasks-backup-$(date +%Y%m%d)

# Upload and extract new version
# From local: scp mergetasks-new.zip mergetasks@YOUR_OVH_IP:~/
unzip -o ~/mergetasks-new.zip -d ~/mergetasks-app

# Restore your .env (if overwritten)
# cp ../mergetasks-backup-YYYYMMDD/.env .

# Install any new dependencies
pnpm install

# Run any new migrations (check the changelog)
# mysql -u mergetasks -p mergetasks < drizzle/0032_whatever.sql

# Rebuild
pnpm build

# Run tests
pnpm test

# Restart
pm2 restart mergetasks

# Verify
curl https://app.mergetasks.com/health
```

Always preserve:

`.env` (your configuration), `uploads/` (user-uploaded files), and the MySQL database (never drop it).

21 — Troubleshooting

"Cannot connect to database"

```
sudo systemctl status mysql
mysql -u mergetasks -p mergetasks -e "SELECT 1;"
# Check DATABASE_URL in .env matches the credentials
```

"SMTP verification failed" / Users can't receive codes

```
node -e "require('nodemailer').createTransport({host:'smtp.gmail.com',port:587,
  secure:false,auth:{user:process.env.SMTP_USER,pass:process.env.SMTP_PASS}})
  .verify().then(()=>console.log('OK')).catch(e=>console.error(e.message))"
```

- Gmail App Passwords require 2-Step Verification enabled
- Some OVH VPS block port 25/587 by default — check firewall rules
- Try port 465 with `SMTP_PORT=465` (implicit TLS)

"Redis connection refused"

```
redis-cli ping
sudo systemctl status redis-server
# If using a password, verify REDIS_URL includes it
```

"502 Bad Gateway" from Nginx

```
pm2 status
pm2 logs mergetasks --lines 50
# Common cause: PORT mismatch between .env and Nginx proxy_pass
```

"Account temporarily locked"

```
mysql -u mergetasks -p mergetasks -e \
  "UPDATE users SET failedLoginAttempts = 0, lockedUntil = NULL WHERE email = 'user@example.com';"
```

SSL certificate renewal

Certbot auto-renews via a systemd timer. Verify:

```
sudo certbot renew --dry-run
```

If using Cloudflare with Full (strict) SSL, the origin certificate must stay valid. Certbot handles this automatically.

22 — Environment Variables Reference

The complete list is in `.env.example`. Here is a summary grouped by criticality.

Required (server will not function without these)

Variable	Example	Purpose
<code>NODE_ENV</code>	<code>production</code>	Enables production optimizations
<code>DATABASE_URL</code>	<code>mysql://user:pass@localhost:3306/mergetasks</code>	MySQL connection
<code>SESSION_SECRET</code>	(64+ hex chars)	JWT signing key
<code>CREDENTIAL_ENCRYPTION_KEY</code>	(64 hex chars)	AES-256 encryption for stored credentials
<code>APP_BASE_URL</code>	<code>https://app.mergetasks.com</code>	Used in email links and redirects
<code>SMTP_HOST</code>	<code>smtp.gmail.com</code>	Email server
<code>SMTP_PORT</code>	<code>587</code>	Email port
<code>SMTP_USER</code>	<code>you@gmail.com</code>	Email username
<code>SMTP_PASS</code>	(app password)	Email password
<code>OPENAI_API_KEY</code>	<code>sk- ...</code>	AI copilot

Required in production

Variable	Example	Purpose
<code>REDIS_URL</code>	<code>redis://localhost:6379</code>	Rate limiting + session revocation

Optional but recommended

Variable	Purpose
<code>STRIPE_SECRET_KEY</code>	Subscription billing
<code>STRIPE_WEBHOOK_SECRET</code>	Stripe webhook verification
<code>SENTRY_DSN</code>	Server error tracking
<code>VITE_SENTRY_DSN</code>	Client error tracking

Optional (feature-specific)

Variable	Purpose
<code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>	Google OAuth login
<code>MICROSOFT_CLIENT_ID</code> / <code>MICROSOFT_CLIENT_SECRET</code>	Microsoft OAuth login
<code>GMAIL_CLIENT_ID</code> / <code>GMAIL_CLIENT_SECRET</code>	Gmail send integration
<code>OUTLOOK_CLIENT_ID</code> / <code>OUTLOOK_CLIENT_SECRET</code>	Outlook send integration
<code>ASI_API_KEY</code> / <code>ASI_ACCOUNT_ID</code>	ASI ESP product search
<code>PS_SANMAR_USERNAME</code> / <code>PS_SANMAR_PASSWORD</code>	SanMar PromoStandards
<code>PS_SS_USERNAME</code> / <code>PS_SS_PASSWORD</code>	S&S Activewear PromoStandards
<code>PS_ALPHABRODER_USERNAME</code> / <code>PS_ALPHABRODER_PASSWORD</code>	alphabroder PromoStandards

23 — Migration Files Reference

All migrations live in `drizzle/`. They are numbered sequentially and must be applied in order.

Migration	Description
<code>0000</code> – <code>0024</code>	Core schema: users, clients, products, proposals, orders, stores, etc.
<code>0025</code>	Multi-tenancy: organizations, org members, org invites
<code>0026</code>	External product catalog columns (ASI, PromoStandards)
<code>0027</code>	Initial FK constraints batch
<code>0028</code>	PCI audit log table
<code>0029</code>	Account lockout columns, OTP hash widening, audit timestamp fix
<code>0030</code>	32 remaining FK constraints for full referential integrity
<code>0031</code>	SSO: <code>store_identity_providers</code> table + SSO columns on <code>store_users</code>

For a fresh deployment, `pnpm db:push` handles 0000–0028 automatically. Then run 0029, 0030, and 0031 manually as shown in Section 8.